

Documentação Técnica — Loja do Sr. Campos

1. Stack Tecnológica

Camada	Tecnologia
Frontend	ASP.NET MVC 5.2.9 (.NET Framework 4.8)
Backend	ASP.NET Web API 2 — 5.2.7 (.NET Framework 4.8)
Views	Razor (CSHTML) + Bootstrap 5.2.3 + jQuery 3.7.0
ORM	Entity Framework 6.5.2 (Code First Migrations)
Banco de dados	SQL Server (LocalDB / Express)
Mediator / CQRS	MediatR 9.0.0
Validação	FluentValidation 11.12.0

2. Arquitetura

A solução é composta por cinco projetos organizados em camadas seguindo os princípios de Clean Architecture, mais um projeto de testes:

TesteCamposDealer.Web — Frontend MVC (ASP.NET MVC 5)

TesteCamposDealer.API — Backend REST (ASP.NET Web API 2)

TesteCamposDealer.Application — Lógica de aplicação (CQRS)

TesteCamposDealer.Infrastructure — Persistência (EF6)

TesteCamposDealer.Domain — Núcleo do domínio

TesteCamposDealer.Tests — Testes unitários (xUnit)

O **Web** é a camada de apresentação: o browser acessa o frontend MVC, que delega chamadas HTTP para a API REST via ApiClient. Os controllers MVC não acessam o banco diretamente — todo acesso a dados passa pela API.

A **API** recebe as requisições REST, mapeia para ViewModels e despacha commands/queries via MediatR. Os **Handlers** (camada Application) executam a lógica de negócio e se comunicam com o banco através das interfaces de repositório

definidas no **Domain**. A **Infrastructure** implementa essas interfaces com Entity Framework 6 Code First.

Fluxo de uma requisição (ex.: criar venda):

1. Web controller chama `ApiClient.PostAsync("api/vendas", ...)`
2. `VendaController` (API) recebe o request e dispara `CreateVendaCommand` via MediatR
3. `ValidationBehavior` valida o command antes de chegar ao handler
4. `CreateVendaHandler` aplica regras de negócio, persiste via `IVendaRepository`
5. `VendaRepository` (Infrastructure) executa a operação no banco via EF6
6. A resposta sobe em cadeia até o Web controller, que renderiza a View

2.1 Padrão CQRS com MediatR

Todos os casos de uso são implementados como `IRequest<T>` processados por `IRequestHandler<TRequest, TResponse>`. Isso separa leitura de escrita e isola a lógica de negócio dos controllers:

- Commands: `CreateClienteCommand`, `UpdateProdutoCommand`, `CreateVendaCommand`, etc.
- Queries: `GetAllVendasQuery`, `GetRankingQuery`, `GetVendasByClienteQuery`, etc.

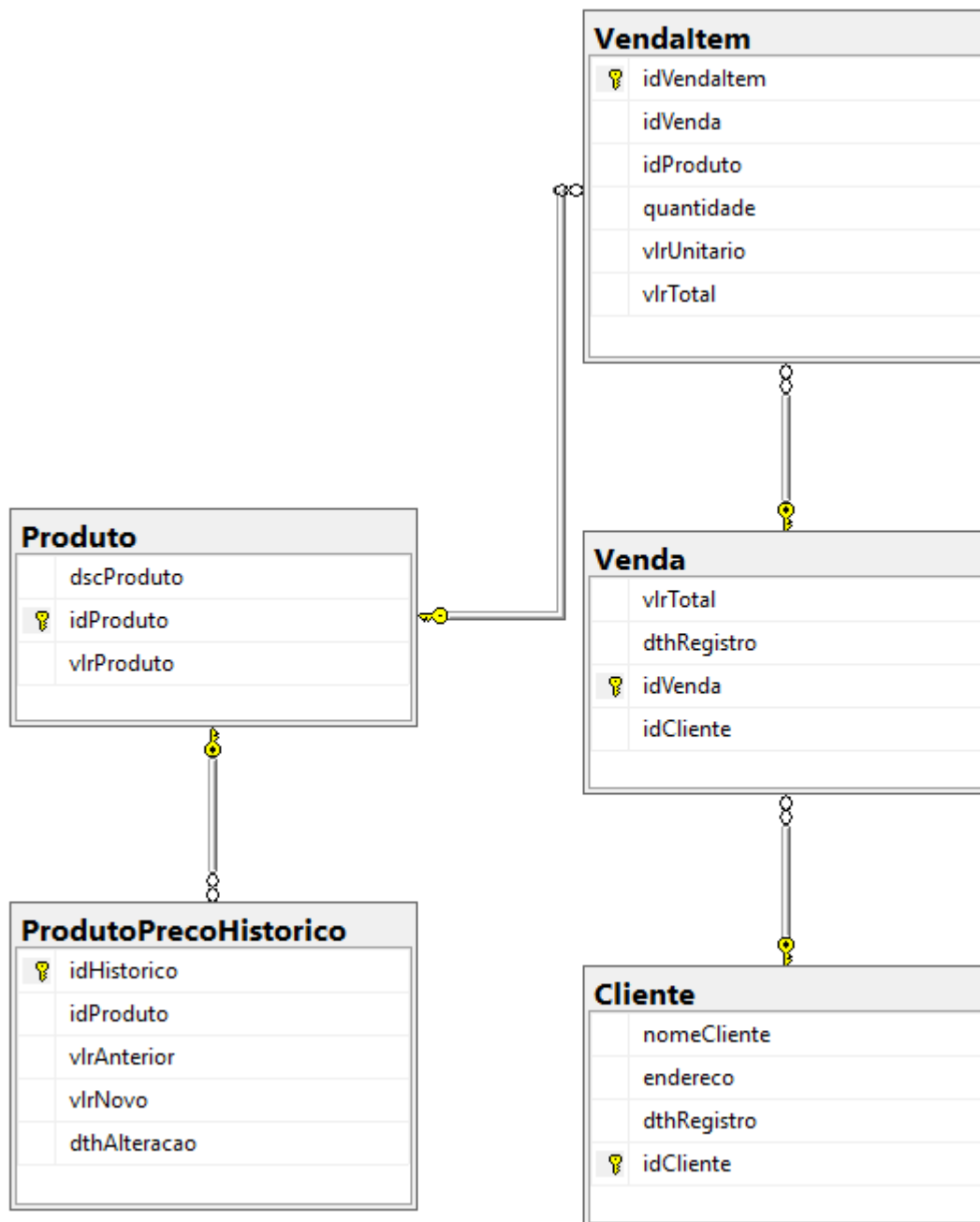
2.2 Unit of Work + Repository

O `IUnitOfWork` agrupa os três repositórios (`IClienteRepository`, `IProdutoRepository`, `IVendaRepository`) e expõe um único `CommitAsync()`. Garante que alterações em múltiplas entidades sejam persistidas em uma única transação.

3. Modelagem de Dados

3.1 Diagrama de Entidades

Diagrama de Entidades



Cliente

└─ idCliente UNIQUEIDENTIFIER (PK)
└─ nomeCliente VARCHAR(200)
└─ endereco VARCHAR(500)
└─ dthRegistro DATETIME

Produto

└─ idProduto UNIQUEIDENTIFIER (PK)
└─ dscProduto NVARCHAR(200)
└─ vlrProduto DECIMAL(18,2)
└─ ProdutoPrecoHistorico (1:N)

ProdutoPrecoHistorico

└─ idHistorico UNIQUEIDENTIFIER (PK)
└─ idProduto UNIQUEIDENTIFIER (FK → Produto)
└─ vlrAnterior DECIMAL(18,2)
└─ vlrNovo DECIMAL(18,2)
└─ dthAlteracao DATETIME

Venda

└─ idVenda UNIQUEIDENTIFIER (PK)
└─ idCliente UNIQUEIDENTIFIER (FK → Cliente)
└─ vlrTotal DECIMAL(18,2)
└─ dthRegistro DATETIME
└─ VendaItem (1:N)

VendaItem

└─ idVendaItem UNIQUEIDENTIFIER (PK)
└─ idVenda UNIQUEIDENTIFIER (FK → Venda)
└─ idProduto UNIQUEIDENTIFIER (FK → Produto)
└─ quantidade INT
└─ vlrUnitario DECIMAL(18,2) ← preço capturado no momento da venda

└─ vlrTotal DECIMAL(18,2) ← quantidade × vlrUnitario

3.2 Decisão: DECIMAL em vez de FLOAT

O banco legado usava FLOAT para valores monetários, causando erros de arredondamento ao materializar no tipo decimal do C#. Todos os campos de valor foram migrados para DECIMAL(18,2) via migration EF6.

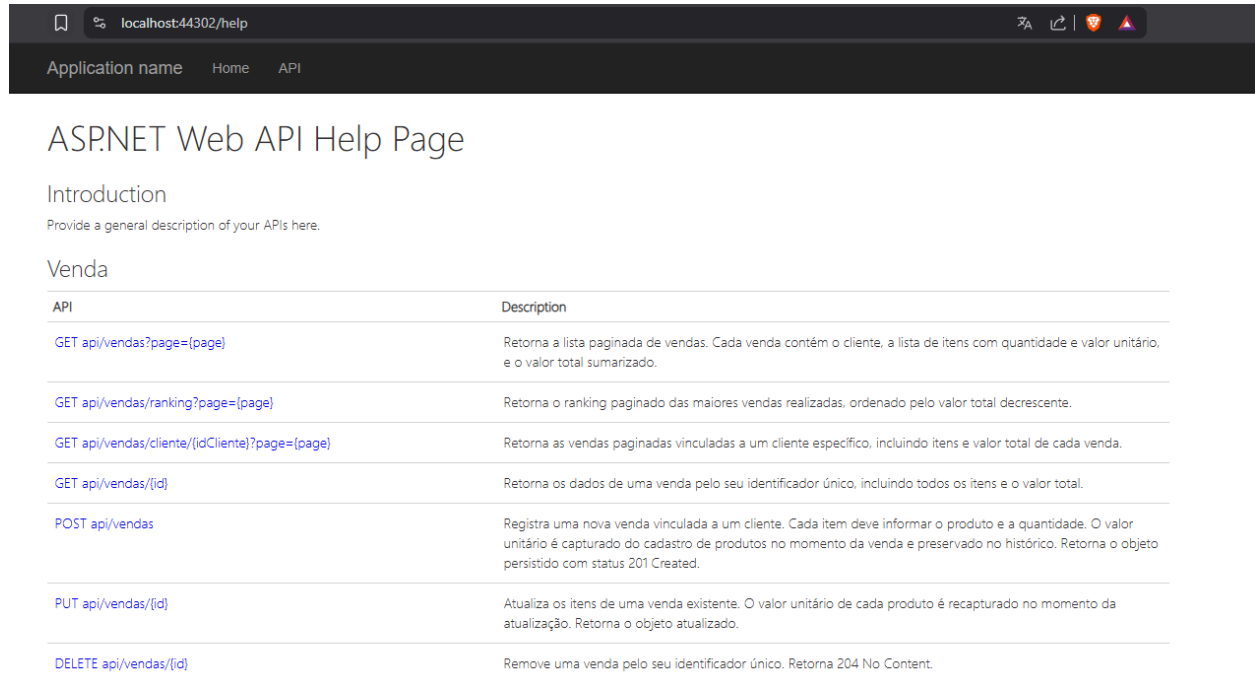
3.3 Decisão: UUID v7 em vez de INT auto-incremento

O esquema legado usava INT como PK. A evolução utilizou UNIQUEIDENTIFIER com UUID v7 (biblioteca Medo.Uuid7), que:

- Gera IDs ordenados cronologicamente, eliminando fragmentação de índice no SQL Server
- Permite geração de IDs na aplicação sem round-trip ao banco
- Usa NewMsSqlUniqueIdentifier() que reordena os bytes para o critério de ordenação do SQL Server

4. Rotas

Todas as rotas podem ser encontradas na collection do Postman, na pasta 'docs', na raiz do projeto. Também é possível acessar a documentação das rotas em 'https://localhost:44302/help', enquanto o back end estiver rodando.



The screenshot shows a web browser window with the address bar at 'localhost:44302/help'. The page title is 'ASP.NET Web API Help Page'. Below the title, there is an 'Introduction' section with the text 'Provide a general description of your APIs here.' followed by a section titled 'Venda'. This section contains a table with two columns: 'API' and 'Description'.

API	Description
GET api/vendas?page={page}	Retorna a lista paginada de vendas. Cada venda contém o cliente, a lista de itens com quantidade e valor unitário, e o valor total sumarizado.
GET api/vendas/ranking?page={page}	Retorna o ranking paginado das maiores vendas realizadas, ordenado pelo valor total decrescente.
GET api/vendas/cliente/{idCliente}?page={page}	Retorna as vendas paginadas vinculadas a um cliente específico, incluindo itens e valor total de cada venda.
GET api/vendas/{id}	Retorna os dados de uma venda pelo seu identificador único, incluindo todos os itens e o valor total.
POST api/vendas	Registra uma nova venda vinculada a um cliente. Cada item deve informar o produto e a quantidade. O valor unitário é capturado do cadastro de produtos no momento da venda e preservado no histórico. Retorna o objeto persistido com status 201 Created.
PUT api/vendas/{id}	Atualiza os itens de uma venda existente. O valor unitário de cada produto é recapturado no momento da atualização. Retorna o objeto atualizado.
DELETE api/vendas/{id}	Remove uma venda pelo seu identificador único. Retorna 204 No Content.

Clientes

Método	Rota	Ação
GET	api/Cliente	Lista paginada
GET	api/Cliente/{id}	Consulta por id
POST	api/Cliente	Formulário de criação
PUT	api/Cliente/{id}	Persiste alterações
DELETE	api/Cliente/{id}	Remove cliente

Produtos

Método	Rota	Ação
GET	api/produtos	Lista paginada
GET	api/produtos /{id}	Consulta por id
POST	api/produtos	Formulário de criação
PUT	api/produtos /{id}	Persiste alterações
DELETE	api/produtos /{id}	Remove produto

Vendas

Método	Rota	Ação
GET	api/vendas	Lista paginada
GET	api/vendas /{id}	Consulta por id
POST	api/vendas	Formulário de criação
PUT	api/vendas /{id}	Persiste alterações
DELETE	api/vendas /{id}	Remove venda
GET	api/vendas/ranking	Consulta ranking de vendas
GET	api/vendas/cliente/{clienteId}	Consulta vendas ligadas a um cliente

5. Decisões Técnicas Relevantes

.NET Framework 4.6 → 4.8

O 4.8 é a versão LTS final do .NET Framework, com suporte estendido da Microsoft e sem impacto de compatibilidade sobre os demais pacotes.

LINQ to SQL → Entity Framework 6

O banco legado era acessado via LINQ to SQL (.dbml com classes auto-geradas). Essa abordagem tem três limitações críticas para o escopo do projeto: não suporta migrations (o schema precisaria ser mantido manualmente), não suporta async/await nativo (todos os acessos são síncronos e bloqueantes), e o mapeamento de relacionamentos 1:N aninhados (Venda → VendaItem → Produto) é verbose e frágil. A migração para EF6 Code First resolveu os três pontos e ainda habilitou o padrão Repository + Unit of Work testável com Moq.

Lazy Loading desabilitado

O ApplicationDbContext desabilita proxy e lazy loading. Todas as queries que precisam de navegação usam Include() explícito. Isso evita o problema N+1 e torna o comportamento previsível em contextos assíncronos.

6. Autoavaliação Técnica

Backend

Tenho experiência sólida no desenvolvimento de APIs utilizando .NET 8 (ou superior) e também com aplicações em ASP.NET MVC (.NET Framework 4.8). Sou capaz de estruturar, manter e evoluir APIs sem esforço.

Em persistência de dados, possuo experiência com Entity Framework Core e Entity Framework, além de conhecimento em SQL Server demais tipos de banco relacionais. Também aplico boas práticas de logging e tratamento de exceções para garantir observabilidade e resiliência das aplicações.

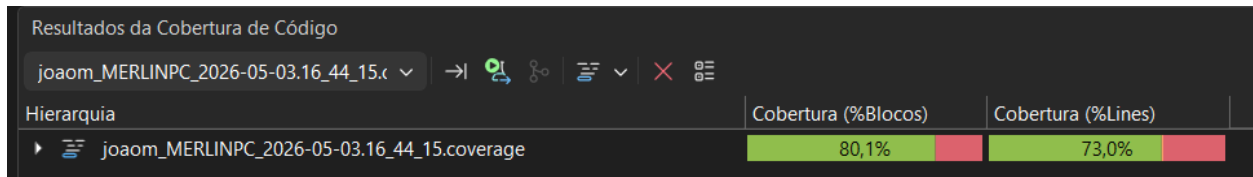
Frontend

Tenho experiência com desenvolvimento frontend utilizando Angular e React (com TypeScript), além de páginas Razor em projetos ASP.NET MVC. Consigo manter e evoluir interfaces existentes, garantindo integração adequada com o backend e boa organização do código.

Testes

Possuo boa familiaridade com testes de integração, validando o funcionamento de endpoints e regras de negócio por meio de chamadas estruturadas e documentadas no Postman. Neste projeto, todos os endpoints foram documentados (arquivo **CamposDealer.postman_collection.json**), garantindo rastreabilidade e facilidade de

validação. Além disso, a cobertura de testes supera 80% (conforme evidenciado abaixo), reforçando a confiabilidade das implementações.



Ferramentas

Tenho domínio das principais ferramentas de desenvolvimento, incluindo Visual Studio e SQL Server Management Studio. Utilizo Postman e Insomnia para validação de fluxos HTTP e testes de APIs, além de Git para controle de versão e colaboração em equipe.

Processos

Possuo familiaridade com a metodologia SCRUM, participando ativamente de cerimônias como planejamento de sprints, refinamento e reviews. Tenho experiência com testes unitários e integrados, depuração de código utilizando breakpoints, além de atuação na correção de bugs e análise de problemas em produção com apoio de ferramentas de telemetria.